Course Name:	Web Development Laboratory (316U01L306)	**Semester:**	**III**
Date of Performance:	___ / ___ / _____	**DIV/ Batch No:**	
Student Name:		**Roll No:**	

Experiment No: 5

Title: JavaScript Programming for Climate Intelligence – Heatwave Monitoring, Prediction, and Early Warning

Aim of the Experiment: To develop JavaScript programs for a Climate Intelligence system that monitors heatwave conditions, analyzes environmental information, determines heatwave risk, and provides early-warning recommendations using JavaScript concepts such as variables, operators, expressions, control statements, objects, functions, arrays, strings, and regular express

Objectives for the Experiment:

- To use JavaScript variables, values, operators, and expressions for processing climate-related information.
- To apply control statements for determining heatwave risk and generating appropriate warnings.
- To create JavaScript objects representing climate monitoring information.
- To use functions for performing climate-related calculations and generating recommendations.
- To use arrays for storing and analyzing temperature and environmental readings.
- To use strings and regular expressions for validating location and user information.
- To integrate JavaScript with HTML forms for a Climate Intelligence early-warning application.

COs to be achieved:

CO1: Developing webpages using HTML, CSS and JavaScript.

Books/ Journals/ Websites references:

1. MDN Web Docs – JavaScript Guide and HTML Forms.
2. W3Schools – JavaScript.

Theory:

Task 1: Heatwave Temperature Analysis

Problem Statement

Develop a JavaScript program to accept the current temperature and threshold temperature of a location and determine the heatwave condition.

Functional Requirements

Declare variables for:

- Location name
- Current temperature
- Heatwave threshold temperature
- Humidity
- Monitoring date

Perform the following operations:

1. Calculate the difference between the current temperature and heatwave threshold.
2. Determine whether the current temperature has crossed the threshold.
3. Display the heatwave monitoring status.
4. Display the location and temperature information.
5. Use JavaScript message-printing methods such as:
    - alert()
    - document.write()
    - console.log()

Expected Output

Location: Mumbai
Current Temperature: 39°C
Heatwave Threshold: 37°C
Temperature Difference: 2°C

Status: Heatwave Condition Detected

Concepts Tested

- Values
- Variables

- Data types
- Operators
- Expressions
- Arithmetic operations
- Message printing
- Basic conditional statement

Deliverable

heatwave_analysis.html

Task 2: Heatwave Risk and Early-Warning Decision

Problem Statement

Develop a JavaScript program that recommends the appropriate heatwave warning level based on temperature, humidity, and forecast conditions.

Input Parameters

- Current temperature
- Humidity
- Forecast temperature
- Heatwave condition
- Vulnerable population alert requirement

Decision Rules

Condition	Recommendation
Temperature ≥ 40°C	**Severe Heatwave Warning**
Temperature ≥ 38°C and Humidity ≥ 60%	**High Heat Risk – Issue Warning**
Temperature between 35°C and 38°C	**Moderate Heat Risk – Monitor Conditions**
Forecast temperature ≥ 40°C	**Early Warning – Prepare for Heatwave**
Temperature < 35°C	**Normal – Continue Monitoring**

Requirements

1. Accept climate values using JavaScript variables.
2. Use if, else-if, and else statements.
3. Use relational operators such as >, <, >=, and <=.
4. Use logical operators such as && and ||.
5. Display the appropriate heatwave risk level.

6. Display an early-warning recommendation.

Example Output

Location: Pune
Temperature: 39°C
Humidity: 65%

Risk Level: HIGH
Recommendation: Issue Heatwave Warning

Concepts Tested

- Variables
- Operators
- Relational operators
- Logical operators
- Expressions
- if-else-if
- Control statements
- Message printing

Deliverable

heatwave_warning.html

Task 3: Create a JavaScript Object for Climate Monitoring

Problem Statement

Create a JavaScript object representing a location monitored by the Climate Intelligence system.

Object Properties

The climate monitoring object should contain:

- Location name
- City
- Temperature
- Humidity
- Wind speed
- Forecast temperature
- Heatwave risk level

- Alert status
- Monitoring date

Required Methods

Create methods to:

1. Display complete climate information.
2. Calculate the temperature difference from the threshold.
3. Determine the heatwave risk level.
4. Display the early-warning status.
5. Update the alert status.

Example Object

```
let climateData = {
   location: "Mumbai",
   temperature: 39,
   humidity: 65,
   windSpeed: 12,
   forecastTemperature: 41,
   riskLevel: "High",
   alertStatus: "Active",

   displayData: function() {
      console.log("Location: " + this.location);
      console.log("Temperature: " + this.temperature + "°C");
      console.log("Humidity: " + this.humidity + "%");
      console.log("Risk Level: " + this.riskLevel);
   }
};
```

Requirements

1. Create the object using object-literal syntax.
2. Access object properties using the dot operator.
3. Create functions/methods inside the object.
4. Use this to access object properties.
5. Display the climate information on the webpage or console.

Concepts Tested

- Object-oriented JavaScript
- Objects
- Properties

- Methods
- Functions
- this keyword
- Conditional statements

Deliverable

climate_object.html

Task 4: Analyze Temperature Readings Using Arrays

Problem Statement

A Climate Intelligence system collects temperature readings at regular intervals. Store the readings in a JavaScript array and analyze the temperature conditions.

Consider the following temperature readings:

let temperatureReadings = [34, 36, 38, 39, 41, 40, 37, 35, 39, 42];

Perform the Following Operations

1. Display all temperature readings.
2. Find the minimum temperature.
3. Find the maximum temperature.
4. Calculate the average temperature.
5. Count the number of readings above the heatwave threshold.
6. Count the number of readings equal to or above 40°C.
7. Display all readings that indicate a high heat condition.

Example Output

Temperature Readings:
34 36 38 39 41 40 37 35 39 42

Minimum Temperature: 34°C
Maximum Temperature: 42°C
Average Temperature: 38.1°C
Readings ≥ 40°C: 3
Heatwave Condition Readings: 41, 40, 42

Concepts Tested

- Arrays
- Array traversal
- Loops
- Array methods
- Functions
- Conditional statements
- Mathematical calculations

Deliverable

temperature_analysis.html

Task 5: Validate Climate Monitoring Registration Details

Problem Statement

Develop a JavaScript program to validate the details entered by a user for registering with the Climate Intelligence Heatwave Early-Warning System.

Fields to Validate

- User name
- Mobile number
- Email address
- Location name
- User ID
- PIN/Postal Code

Validation Rules

Field	Validation Rule
User Name	Only alphabets and spaces
Mobile Number	Exactly 10 digits
Email	Valid email format
Location	Alphabets and spaces
User ID	Format: CLM-1234
PIN Code	Exactly 6 digits

Example

User Name: Rahul Sharma

Mobile Number: 9876543210
Email: rahul@gmail.com
Location: Mumbai
User ID: CLM-1234
PIN Code: 400077

Requirements

1. Accept user details using JavaScript.
2. Validate the user name.
3. Validate the mobile number.
4. Validate the email address.
5. Validate the location.
6. Validate the Climate Intelligence User ID.
7. Display appropriate error messages.
8. Display a success message when all fields are valid.

Regular Expression Examples

User Name:

/^[A-Za-z]+$/

Mobile Number:

/^[6-9][0-9]{9}$/

Climate User ID:

/^CLM-[0-9]{4}$/

PIN Code:

/^[0-9]{6}$/

Concepts Tested

- Strings
- String methods
- Regular expressions
- test()
- Functions
- Boolean expressions
- Conditional statements

Deliverable

climate_validation.html

Task 6: Develop a Climate Intelligence Registration and Early-Warning Form

Problem Statement

Develop an HTML-based registration form for users who want to access the **Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning System**.

The form should collect user, location, and alert-preference information and validate the entered data using JavaScript.

Input Fields

Create a registration form containing:

- User Name
- Mobile Number
- Email
- User ID
- Location/City
- PIN/Postal Code
- Age Group
- User Category
- Preferred Alert Channel
- Registration Date

Suggested User Categories

- General Public
- Senior Citizen
- Student
- Outdoor Worker
- Healthcare Worker
- Government/Disaster Management
- Other

Validation Requirements

1. No mandatory field should remain empty.
2. User name must contain only alphabets and spaces.

3. Mobile number must contain exactly 10 digits.
4. Email address must follow a valid format.
5. User ID must follow the pattern:

CLM-1234

6. PIN/Postal Code must contain exactly six digits.
7. User Category must be selected.
8. Preferred Alert Channel must be selected.
9. Location/City must not be empty.
10. Registration date must be entered.
11. Registration date should not be a future date.
12. Appropriate error messages should be displayed beside invalid fields.
13. Invalid fields should be highlighted.
14. The form should not be submitted until all validation rules are satisfied.
15. After successful validation, display:

Registration Successful!

You are registered for Climate Intelligence
Heatwave Monitoring and Early-Warning Alerts.

HTML Elements to Use

Students should use:

- <form>
- <label>
- <input>
- <select>
- <option>
- <fieldset>
- <legend>
- <button>

JavaScript Requirements

The program should use JavaScript to:

1. Access HTML form elements using the DOM.
2. Read values entered by the user.
3. Validate the input values.

4. Use functions for individual validation operations.
5. Use regular expressions for structured fields.
6. Use conditional statements.
7. Handle the form submit event.
8. Display validation errors.
9. Prevent invalid form submission.
10. Display a success message after valid registration.

Code:

Expected Output / Screenshots:(Paste here):

Conclusion and Discussion:

Students should write the conclusion in their own words based on the work performed.